

Data Analysis using SQL (Milestone 2)

Analysis – I

Q1. Round the average payment values to integer (no decimal) for each payment type and display the results sorted in ascending order.

```
SELECT payment_type,round(AVG(payment_value), 0) AS rounded_avg_payment
FROM amazon_analysis.payments
GROUP BY payment_type
ORDER BY rounded_avg_payment ASC
```

	payment_type character varying 🔒	rounded_avg_payment numeric 🔒
1	not_defined	0
2	voucher	66
3	debit_card	143
4	boleto	145
5	credit_card	163

Q2. Calculate the percentage of total orders for each payment type, rounded to one decimal place, and display them in descending order.

```
select payment_type,round(count(*) * 100.0 / (select count(*) from amazon_analysis.payments),1)
as percentage_orders
from amazon_analysis.payments
group by payment_type
order by percentage_orders desc
```

	payment_type character varying 🔒	percentage_orders numeric 🔒
1	credit_card	73.9
2	boleto	19.0
3	voucher	5.6
4	debit_card	1.5
5	not_defined	0.0

Q3. Identify all products priced between 100 and 500 BRL that contain the word 'Smart' in their name. Display these products, sorted by price in descending order.

```
select p.product_id,p.product_category_name,oi.price
from amazon_analysis.products p
join amazon_analysis.order_items oi
on p.product_id = oi.product_id
where oi.price between 100 and 500
and p.product_category_name ilike '%Smart%'
order by oi.price desc
```

	product_id character varying	product_category_name character varying	price numeric (10,2)
1	1df1a2df8ad2b9d3aa49fd851e3145...	smart	440.00
2	7debe59b10825e89c1cbcc8b190c8...	smart	350.00
3	ca86b9fe16e12de698c955aedff0ae...	smart	349.00
4	ca86b9fe16e12de698c955aedff0ae...	smart	349.00
5	0e52955ca8143bd179b311cc454a6...	smart	335.00
6	7aeaa8f3e592e380c420e8910a717...	smart	330.00
7	7aeaa8f3e592e380c420e8910a717...	smart	330.00
Total rows: 37		Query complete 00:00:00.120	

Q4. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

```
select date_trunc('month', o.order_purchase_timestamp) as month,
round(sum(oi.price)) as total_sales
from amazon_analysis.orders o
join amazon_analysis.order_items oi
on o.order_id = oi.order_id
group by month
Order by total_sales DESC
limit 3
```

	month timestamp without time zone	total_sales numeric
1	2017-11-01 00:00:00	1010705
2	2018-04-01 00:00:00	996982
3	2018-05-01 00:00:00	996839

Q5. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.

```
select p.product_category_name,
max(oi.price) as max_price,
min(oi.price) as min_price,
max(oi.price) - min(oi.price) as price_difference
from amazon_analysis.products p
join amazon_analysis.order_items oi
on p.product_id = oi.product_id
group by p.product_category_name
having max(oi.price) - min(oi.price) > 500
order by price_difference desc
```

	product_category_name character varying	max_price numeric	min_price numeric	price_difference numeric
1	utilidades_domesticas	6735.00	3.00	6732.00
2	pcs	6729.00	35.00	6694.00
3	artes	6499.00	4.00	6495.00
4	eletroportateis	4799.00	7.00	4792.00
5	instrumentos_musicais	4400.00	5.00	4395.00
6	consoles_games	4100.00	5.00	4095.00
7	esporte_lazer	4059.00	5.00	4054.00
Total rows: 57		Query complete 00:00:00.122		

Q6. Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.

```
select payment_type, round(STDDEV(payment_value), 2) AS std_deviation
from amazon_analysis.payments
group by payment_type
order by std_deviation asc
```

	payment_type character varying	std_deviation numeric
1	not_defined	0.00
2	voucher	115.52
3	boleto	213.58
4	credit_card	222.12
5	debit_card	245.79

Q7. Retrieve the list of products where the product category name is missing or contains only a single character.

```
select product_id, product_category_name
from amazon_analysis.products
where product_category_name is null
or length(trim(product_category_name)) <= 1
```

	product_id [PK] character varying	product_category_name character varying
1	a41e356c76fab66334f36de622ecbd...	[null]
2	d8dee61c2034d6d075997acef1870...	[null]
3	56139431d72cd51f19eb9f7dae4d16...	[null]
4	46b48281eb6d663ced748f324108c...	[null]
5	5fb61f482620cb672f5e586bb132ea...	[null]
6	e10758160da97891c2fdc35f0f03...	[null]
7	39e3b9b12cd0bf8ee681bbc1c130fe...	[null]
Total rows: 614		Query complete 00:00:00.111

Analysis – II

Q1. Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count.

```
select case when payment_value < 200 then 'Less than 200 BRL'
when payment_value between 200 AND 1000 then '200 - 1000 BRL'
else 'Over 1000 BRL'
end as order_range, payment_type, count (*) as payment_count
from amazon_analysis.payments
group by order_range, payment_type
order by payment_count DESC
```

	order_range text	payment_type character varying	payment_count bigint
1	Less than 200 B...	credit_card	60548
2	Less than 200 B...	boleto	16444
3	200 - 1000 BRL	credit_card	15303
4	Less than 200 B...	voucher	5476
5	200 - 1000 BRL	boleto	3162
6	Less than 200 B...	debit_card	1287
7	Over 1000 BRL	credit_card	944
Total rows: 13		Query complete 00:00:00.096	

Q2. Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.

```
select p.product_category_name,
min(oi.price) as min_price,
max(oi.price) as max_price,
round(avg(oi.price), 2) as avg_price
from amazon_analysis.products p
join amazon_analysis.order_items oi
on p.product_id = oi.product_id
group by p.product_category_name
order by avg_price desc
```

	product_category_name character varying	min_price numeric	max_price numeric	avg_price numeric
1	pcs	35.00	6729.00	1098.35
2	portateis_casa_forno_e_cafe	10.00	2899.00	624.30
3	eletrodomesticos_2	14.00	2350.00	476.16
4	agro_industria_e_comercio	13.00	2990.00	341.73
5	instrumentos_musicais	5.00	4400.00	281.67
6	eletroportateis	7.00	4799.00	280.81
7	portateis_cozinha_e_preparadores_de_aliment...	17.00	1099.00	264.73
Total rows: 79		Query complete 00:00:00.291		

Q3. Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.

```
select c.customer_unique_id, count(o.order_id) as total_orders
from amazon_analysis.customers c
join amazon_analysis.orders o
on c.customer_id = o.customer_id
group by c.customer_unique_id
having count(o.order_id) > 1
order by total_orders desc
```

	customer_unique_id character varying	total_orders bigint
1	a91e80fbe80ddc07de66a5cf927029...	16
2	a6168cd79131e64acef92e3c74d6cc...	16
3	363f980585bf04c1a88fdb986011c5...	16
4	cbd0350d4ccba9772e8e768d4a4a5...	16
5	417b909c0962b2610f1cfeb1c14789...	16
6	5f94af52aef02c968a2e0f01f430864e	16
7	1b6d29725255a77667a8c639eeb4c...	16
Total rows: 3140		Query complete 00:00:00.903

Q4. Use a temporary table to define these categories and join it with the customers table to update and display the customer types.

```
with cte as (
select c.customer_id,c.customer_unique_id,
count(o.order_id) as total_orders
from amazon_analysis.customers c
join amazon_analysis.orders o
on c.customer_id = o.customer_id
group by c.customer_id, c.customer_unique_id
)
select customer_unique_id,total_orders,
case when total_orders = 1 then 'New'
when total_orders between 2 and 4 then 'Returning'
else 'Loyal'
end as customer_type
from cte
order by total_orders desc
```

	customer_unique_id character varying	total_orders bigint	customer_type text
1	1b6d29725255a77667a8c639eeb4c...	16	Loyal
2	96fd69e8b0df76a9a807b01dc82bef...	16	Loyal
3	f9c4e8531c2fe4159beb562fd7c2bd...	16	Loyal
4	3d364a7768fae99678635c4370295...	16	Loyal
5	5f94af52aef02c968a2e0f01f430864e	16	Loyal
6	a6168cd79131e64acef92e3c74d6cc...	16	Loyal
7	cbd0350d4ccba9772e8e768d4a4a5...	16	Loyal
Total rows: 98348		Query complete 00:00:01.361	

Q5. Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.

```
select p.product_category_name, round(sum(oi.price)) as total_revenue
from amazon_analysis.products p
join amazon_analysis.order_items oi
on p.product_id=oi.product_id
group by p.product_category_name
order by total_revenue desc
limit 5
```

	product_category_name  character varying	total_revenue  numeric
1	beleza_saude	1258135
2	relogios_presentes	1203268
3	cama_mesa_banho	1033005
4	esporte_lazer	986264
5	informatica_acessorios	910994

Analysis – III

Q1. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.

```
select season,round(sum(total_sales)) as total_sales
from
(
select case
when extract(month from o.order_purchase_timestamp) in (3,4,5) then 'Spring'
when extract(month from o.order_purchase_timestamp) in (6,7,8) then 'Summer'
when extract(month from o.order_purchase_timestamp) in (9,10,11) then 'Autumn'
else 'Winter'
end as season,
oi.price as total_sales
from amazon_analysis.orders o
JOIN amazon_analysis.order_items oi
ON o.order_id = oi.order_id
)
as seasonal_sales
group by season
order by total_sales desc
```

	season 	total_sales 
	text	numeric
1	Spring	4218339
2	Summer	4121920
3	Winter	2906980
4	Autumn	2349750

Q2. Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

```

SELECT p.product_id,
count(oi.order_item_id) as total_quantity_sold
from amazon_analysis.products p
join amazon_analysis.order_items oi
on p.product_id = oi.product_id
group by p.product_id
having count (oi.order_item_id) > (
    select avg (product_quantity)
    from (
        select count(order_item_id) as product_quantity
        from amazon_analysis.order_items
        group by product_id
    ) as avg_table
)
order by total_quantity_sold desc

```

	product_id [PK] character varying 	total_quantity_sold bigint 
1	aca2eb7d00ea1a7b8ebd4e6831466...	527
2	99a4788cb24856965c36a24e339b6...	488
3	422879e10f46682990de24d770e7f...	484
4	389d119b48cf3043d311335e499d9...	392
5	368c6c730842d78016ad823897a37...	388
6	53759a2ecddad2bb87a079a1f1519f...	373
7	d1c427060a0f73f6b889a5c7c61f2a...	343
Total rows: 6366		Query complete 00:00:00.354

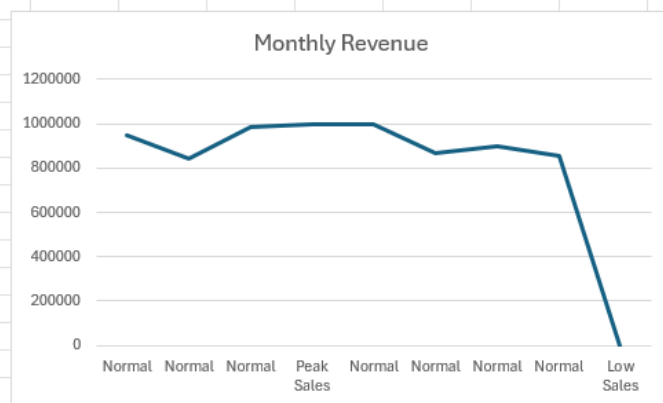
Q3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018). Run a query to calculate total revenue generated each month and identify periods of peak and low sales.

Export the data to Excel and create a graph to visually represent revenue changes across the months.

```
with cte as (  
  select date_trunc('month', o.order_purchase_timestamp) as month,  
         sum(oi.price) as total_revenue  
  from amazon_analysis.orders o  
  join amazon_analysis.order_items oi  
  on o.order_id = oi.order_id  
  where extract(year from o.order_purchase_timestamp) = 2018  
  group by date_trunc('month', o.order_purchase_timestamp)  
)  
select to_char(month, 'YYYY-MM') as month,  
       round(total_revenue, 0) as total_revenue,  
       case  
  when total_revenue = (select max(total_revenue) from cte)  
  then 'Peak Sales'  
  when total_revenue = (select min(total_revenue) from cte)  
  then 'Low Sales'  
  else 'Normal'  
  end as sales_period  
from cte  
order by month
```

	month text 🔒	total_revenue numeric 🔒	sales_period text 🔒
1	2018-01	950434	Normal
2	2018-02	844525	Normal
3	2018-03	983598	Normal
4	2018-04	996982	Peak Sales
5	2018-05	996839	Normal
6	2018-06	865403	Normal
7	2018-07	895778	Normal
Total rows: 9		Query complete 00:00:00.152	

month	total_revenue	sales_period
2018-01	950434	Normal
2018-02	844525	Normal
2018-03	983598	Normal
2018-04	996982	Peak Sales
2018-05	996839	Normal
2018-06	865403	Normal
2018-07	895778	Normal
2018-08	854971	Normal
2018-09	145	Low Sales



Q4. Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.

```
with customer_orders as (
select c.customer_unique_id,
count(o.order_id) as total_orders
from amazon_analysis.customers c
join amazon_analysis.orders o
on c.customer_id = o.customer_id
group by c.customer_unique_id
),
customer_segments as (
select
case
when total_orders between 1 and 2 then 'Occasional'
when total_orders between 3 and 5 then 'Regular'
else 'Loyal'
end as customer_type
from customer_orders
)
select customer_type,
count(*) as count
from customer_segments
group by customer_type
```

	customer_type text	count bigint
1	Occasional	94635
2	Regular	333
3	Loyal	109

Q5. You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.

```
select customer_unique_id, ROUND(avg_order_value, 2) as avg_order_value,
dense_rank() over (order by avg_order_value desc) as customer_rank
from (
select c.customer_unique_id,
avg(order_value) as avg_order_value
from (
select o.order_id, o.customer_id,
sum(oi.price) as order_value
from amazon_analysis.orders o
join amazon_analysis.order_items oi
on o.order_id = oi.order_id
group by o.order_id, o.customer_id
)
as order_totals
join amazon_analysis.customers c
on order_totals.customer_id = c.customer_id
group by c.customer_unique_id
)
as customer_avg
order by avg_order_value desc
limit 20
```

	customer_unique_id character varying	avg_order_value numeric	customer_rank bigint
1	0a0a92112bd4c708ca5fde585afaa8...	13440.00	1
2	763c8b1c9c68a0229c42c9fc6f662b...	7160.00	2
3	dc4802a71eae9be1dd28f5d788ceb...	6735.00	3
4	459bef486812aa25204be022145ca...	6729.00	4
5	ff4159b92c40ebe40454e3e6a7c35e...	6499.00	5
6	4007669dec559734d6f53e029e360...	5934.00	6
7	eebb5dda148d3893cdaf5b5ca3040...	4690.00	7
8	5d0a2980b292d049061542014e896...	4600.00	8
9	48e1ac109decbb87765a3eade6854...	4590.00	9
Total rows: 20		Query complete 00:00:01.472	

Q6. Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.

```
with recursive monthly_sales as (
select oi.product_id, DATE_TRUNC('month', o.order_purchase_timestamp) as month,
sum(oi.price) as monthly_total
from amazon_analysis.orders o
join amazon_analysis.order_items oi
on o.order_id = oi.order_id
group by oi.product_id, month
),

ordered_sales as (
select product_id,
month, monthly_total,
row_number() over (partition by product_id order by month) as rn
from monthly_sales
),

recursive_sales as (
select product_id, month, monthly_total, monthly_total AS total_sales, rn
from ordered_sales
where rn = 1
union all
select os.product_id, os.month, os.monthly_total, rs.total_sales + os.monthly_total as total_sales, os.rn
from ordered_sales os
join recursive_sales rs
on os.product_id = rs.product_id
and os.rn = rs.rn + 1
)

select product_id, month,
ROUND(total_sales, 2) as total_sales
from recursive_sales
order by product_id, month
```

	product_id character varying	month timestamp without time zone	total_sales numeric
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05-01 00:00:00	102.00
2	00088930e925c41fd95ebfe695fd2655	2017-12-01 00:00:00	130.00
3	0009406fd7479715e4bef61dd91f2462	2017-12-01 00:00:00	229.00
4	000b8f95fcb9e0096488278317764d...	2018-08-01 00:00:00	118.00
5	000d9be29b5207b54e86aa1b1ac54...	2018-04-01 00:00:00	199.00
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12-01 00:00:00	52.00
7	00126f27c813603687e6ce486d909d...	2017-09-01 00:00:00	498.00
8	001305006f1b187d27225e1e470476	2017-10-01 00:00:00	20.00
Total rows: 62176		Query complete 00:00:01.926	

Q7. To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018). Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.

```

with monthly_sales as (
select p.payment_type, DATE_TRUNC('month', o.order_purchase_timestamp) as sale_month,
sum(p.payment_value) as monthly_total
from amazon_analysis.orders o
join amazon_analysis.payments p
on o.order_id = p.order_id
where extract(year from o.order_purchase_timestamp) = 2018
group by p.payment_type, DATE_TRUNC('month', o.order_purchase_timestamp)
)

select payment_type,
TO_CHAR(sale_month, 'YYYY-MM') as sale_month,
ROUND(monthly_total, 0) as monthly_total,
ROUND(
(
(monthly_total - lag(monthly_total) over (
partition by payment_type
order by sale_month
))
/
nullif(
lag(monthly_total) over (
partition by payment_type
order by sale_month
), 0
)
) * 100, 2
) as monthly_change

from monthly_sales
order by payment_type, sale_month;

```

	payment_type 	sale_month 	monthly_total 	monthly_change 
	character varying	text	numeric	numeric
1	boleto	2018-01	204845	[null]
2	boleto	2018-02	183113	-10.61
3	boleto	2018-03	191538	4.60
4	boleto	2018-04	193547	1.05
5	boleto	2018-05	195379	0.95
6	boleto	2018-06	153350	-21.51
7	boleto	2018-07	198041	29.14
8	boleto	2018-08	143806	-27.39
9	credit card	2018-01	868880	[null]
Total rows: 36		Query complete 00:00:00.202		